УЧЕБНОЕ ПОСОБИЕ · ПРАКТИКА НА РУССКОМ

PYTHON
С НУЛЯ

Понять. Попробовать.
Решить самостоятельно.

6 тем / 90 задач
Начальный · Средний · Сложный

Ввод и вычисления / Условия / Циклы
Строки / Списки / Функции

Перед каждой задачей - объяснение и отдельный пример.
Внутри - код для анализа, ошибки для исправления,
понятные условия и проверочные случаи.

Для начинающих · Python 3 · Без готовых решений заданий

Открой папку и начинай

В архиве уже подготовлена папка python_practice: 90 файлов ex01.py-ex90.py, этот PDF, README и настройки отступов. Создавать файлы и перепечатывать исходные данные больше не нужно.

ПЕРВЫЙ ШАГ

Распакуй архив. В VS Code выбери «Файл Открыть папку» и открой именно python_practice. Нужен установленный Python 3; дополнительные библиотеки для этих заданий не требуются.

- Открой PDF и файл с номером нужного задания, например ex04.py.
- В строке # Автор впиши своё имя. Прочитай условие и пример в PDF.
- Допиши разделы # Обработка и # Вывод, затем сохрани файл.
- В терминале VS Code запусти python3 ex04.py. В Windows часто используется python ex04.py или py ex04.py.

ТАК ВЫГЛЯДИТ ЗАГОТОВКА: EX04.PY

```
# Автор: _____________________
# Переменные
age = 16
height_cm = 172.0

# Обработка
# Здесь будет твоё решение.

# Вывод
# Здесь будет твой вывод.
```

Traitement означает «Обработка». В Python комментарии начинаются с #; // - это целочисленное деление. Если пустая заготовка пока ничего не выводит, это нормально.

ОТ ГОТОВЫХ ДАННЫХ К ВВОДУ

Для первой проверки значения уже записаны. Если условие просит ввод, затем замени нужные присваивания на input(), int(input()) или float(input()). Повторяющийся ввод в циклах нужно написать самому. Дробные числа вводи с точкой.

С расширением Python от Microsoft можно выбрать интерпретатор и запускать файл кнопкой Run Python File. Терминальная команда работает и без расширения.

Одна задача - один шаг

Сначала прочитай объяснение. Запусти маленький учебный пример и измени в нём данные. Затем открой готовый файл exNN.py и допиши своё решение по условию.

ЧЕТЫРЕ ФОРМАТА ИЗ ИСХОДНОГО СБОРНИКА

Создать / exNN.py	Дополни заготовку: данные уже есть, обработку и вывод нужно написать. Проверь предложенные случаи.
Исправить / exNN.py	Ошибочный исходник уже в файле. Исправь его и допиши комментарий # Ошибка: ...
Прочитать / exNN.py	Запиши прогноз и ход значений комментариями в разделе # Ответ / прогноз. Затем запусти исходник.
Вопрос / exNN.py	Ответь своими словами в комментариях того же файла. Если нужно, допиши короткий опыт.

УРОВНИ ВНУТРИ КАЖДОЙ ТЕМЫ

1 / начальный / первые 5	Одна основная идея, короткая программа.
2 / средний / следующие 5	Нужно соединить несколько изученных приёмов.
3 / сложный / последние 5	Нужен алгоритм, разбор границ или несколько этапов.

Метка «Основное» помогает выбрать первый маршрут, но не заменяет остальные задания. Перед новой темой желательно закончить её начальный блок и несколько задач среднего уровня.

КАК ПОНЯТЬ, ЧТО ГОТОВО

Для каждой программы проверь: обычный случай; точную границу; минимальный допустимый вход; пустые данные, если они разрешены. Прочитай свой код вслух: объясни, зачем нужна каждая строка.

Готовые списки уже перенесены в файлы: меняй их для проверок. В задачах на исправление ошибки намеренно сохранены; ex20.py не запускается до исправления синтаксиса, ex35.py зацикливается до исправления условия или обновления. Ожидаемые результаты в блоке «Проверь себя» помогают проверить поведение, но не содержат решения программы.

Твой маршрут

Иди по темам сверху вниз: новые задания опираются на уже знакомые конструкции. В каждой теме 15 задач: пять на каждый уровень.

1. Ввод, переменные и вычисления

Задания 01-15 · Начало: страница 5 — **15 задач**

2. Условия: if, elif, else

Задания 16-30 · Начало: страница 21 — **15 задач**

3. Циклы: for и while

Задания 31-45 · Начало: страница 37 — **15 задач**

4. Строки и обработка текста

Задания 46-60 · Начало: страница 53 — **15 задач**

5. Списки и наборы данных

Задания 61-75 · Начало: страница 69 — **15 задач**

6. Функции и сборка программ

Задания 76-90 · Начало: страница 85 — **15 задач**

Памятка по синтаксису - стр. 101. Работа с ошибками - стр. 102. Лист прогресса - стр. 103. Об источниках и границах пособия - стр. 104.

ЕСЛИ ХОЧЕТСЯ НАЧАТЬ СРАЗУ

Быстрый первый круг: реши все задачи с меткой «Основное». Затем вернись к остальным и постепенно дойди до сложных задач. Если не получается - уменьшай вход до простого примера и веди таблицу значений.

ТЕМА 1 / ЗАДАНИЯ 01-15

Ввод, переменные и вычисления

Научись превращать данные в понятный результат.

- Программа выполняет инструкции по порядку. Переменная - имя для значения: count = 3. Знак = записывает значение, а == сравнивает два значения.
- input() читает строку. int() превращает целую запись в число, float() - число с дробной частью. Для дробей вводи точку: 7.5. print() выводит результат.
- Основные типы: int - целое число, float - дробное, str - текст, bool - True или False. type(x) показывает тип значения.
- Операторы: +, -, *, /, //, %, **. Деление / даёт float, // округляет частное вниз, % даёт остаток, ** возводит в степень. Скобки задают порядок вычислений.

ОСНОВНОЙ СИНТАКСИС

```
name = input("Имя: ")
count = int(input("Количество: "))
price = float(input("Цена: "))
total = count * price
print(f"{name}, сумма: {total:.2f}")
```

ЧТО ПРИГОДИТСЯ

int(text)	Преобразовать целую запись в число
float(text)	Преобразовать дробную запись
print(a, b)	Вывести значения через пробел
f"{x:.2f}"	Показать число с двумя знаками после точки
a // b; a % b	Частное вниз и остаток; b не равен 0

ЧАСТЫЕ ОШИБКИ

Не складывай строку из input() с числом. Не называй переменные print, input или int. В Python имя чувствительно к регистру: total и Total - разные имена.

ЗАДАНИЕ 01 / 1 ИЗ 15 В ТЕМЕ

План дня

Начальный Создать / ex01.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

print() выводит текст; несколько аргументов разделяются пробелом. Символ \n внутри строки означает переход на новую строку.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
print("Понедельник")
print("9:00", "Библиотека")
```

ОБРАТИ ВНИМАНИЕ

Кавычки в условии обозначают границы текста; выводить их не нужно.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Выведи ровно четыре строки: «Вторник», «8:30 - Математика», «10:20 - Python», «После уроков - отдых». Используй не больше двух вызовов print().

ПРОВЕРЬ СЕБЯ

- Сверь все четыре строки, пробелы и порядок.
- В коде должно быть не больше двух print().

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 02 / 2 ИЗ 15 В ТЕМЕ

Что делает Python?

Начальный

Вопрос / ex02.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Файл .py хранит инструкции. Интерпретатор Python выполняет их. Ошибка в записи не исправляется автоматически, даже если человеку понятен замысел.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
1  message = "Начинаю учиться"
2  print(message)
```

ОБРАТИ ВНИМАНИЕ

Объяснение важнее угадывания ответа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Для каждого утверждения напиши «верно» или «неверно» и одну причину: 1) print() читает ввод; 2) input() всегда возвращает строку; 3) age и Age - одно имя; 4) комментарий начинается с #; 5) Python сам исправляет неверное имя переменной.

ПРОВЕРЬ СЕБЯ

- Должно быть пять ответов с объяснениями.
- Проверь спорные утверждения короткими экспериментами.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 03 / 3 ИЗ 15 В ТЕМЕ

Таблица значений

Начальный Прочитать / ex03.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Правая часть присваивания вычисляется первой. Затем результат заменяет старое значение слева. Предыдущие значения не сохраняются сами.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
x = 4
x = x + 3
print(x)  # 7
```

ОБРАТИ ВНИМАНИЕ

b не меняется автоматически, когда меняется a.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Не запускай код сразу. Составь таблицу значений a, b и c после каждой строки, затем предскажи вывод. Если переменная ещё не создана, ставь прочерк.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
a = 5
b = a + 2
a = b * 3
b = b - a
c = a + b
print(a, b, c)
```

ПРОВЕРЬ СЕБЯ

- Сначала запиши прогноз, потом запусти код.
- Сверь каждое присваивание, а не только последнюю строку.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 04 / 4 ИЗ 15 В ТЕМЕ

Возраст и рост

Начальный Создать / ex04.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Преобразование меняет тип значения, а не только его вид. f-строка вставляет значения переменных в текст через фигурные скобки.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
boxes_text = "6"
boxes = int(boxes_text)
print(boxes + 1)  # 7
```

ОБРАТИ ВНИМАНИЕ

Рост делится на 100; строку нужно преобразовать до вычислений.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси возраст целым числом и рост в сантиметрах дробным числом. Выведи возраст через 10 лет и рост в метрах с двумя знаками после точки. Вход: возраст >= 0, рост > 0.

ПРОВЕРЬ СЕБЯ

- 16 и 172 возраст 26; рост 1.72 м.
- 10 и 145.5 возраст 20; рост 1.46 м.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 05 / 5 ИЗ 15 В ТЕМЕ

Не тот оператор

Начальный

Исправить / ex05.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Строковый + соединяет текст, числовой + складывает числа. Один и тот же знак работает по-разному в зависимости от типов.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = "4"
b = "2"
print(a + b)       # 42
print(int(a) + int(b))  # 6
```

ОБРАТИ ВНИМАНИЕ

Преобразовать нужно оба значения.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Программа должна складывать два введённых целых числа. Исправь код и добавь комментарий «# Ошибка: ...», объясняющий причину неверного результата.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
a = input("Первое число: ")
b = input("Второе число: ")
print(a + b)
```

ПРОВЕРЬ СЕБЯ

- Ввод 7 и 5 12.
- Ввод -2 и 2 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 06 / 6 ИЗ 15 В ТЕМЕ

Средняя оценка

Средний Исправить / ex06.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Умножение и деление выполняются раньше сложения. Чтобы поделить всю сумму, сначала собери её в скобках.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
cost = (6 + 9) / 3
print(cost)  # 5.0
```

ОБРАТИ ВНИМАНИЕ

Не исправляй результат вручную: формула должна работать для других оценок.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь вычисление среднего трёх оценок. Затем замени фиксированные значения вводом; оценки - числа от 0 до 20. Выводи среднее с двумя знаками после точки.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
n1 = 12
n2 = 14
n3 = 16
average = n1 + n2 + n3 / 3
print(average)
```

ПРОВЕРЬ СЕБЯ

- 12, 14, 16 14.00.
- 0, 10, 20 10.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 07 / 7 ИЗ 15 В ТЕМЕ

Чек на наушники

Средний

Создать / ex07.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для суммы покупки умножают цену на количество. Формат :.2f управляет выводом: 9.5 будет показано как 9.50, без изменения исходного числа.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
price = 3.25
count = 4
print(f"К оплате: {price * count:.2f} евро")
```

ОБРАТИ ВНИМАНИЕ

Точку в дробном вводе нельзя заменять запятой без дополнительной обработки.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси название товара, цену > 0 и целое количество >= 1. Выведи три строки: «Товар: ...», «Количество: ...», «Итого: ... евро». Сумму покажи с двумя знаками.

ПРОВЕРЬ СЕБЯ

- Наушники, 49.90, 2 Итого: 99.80 евро.
- Тетрадь, 2.5, 3 Итого: 7.50 евро.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 08 / 8 ИЗ 15 В ТЕМЕ

Скидка для класса

Средний

Создать / ex08.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Процент от суммы равен sum * percent / 100. Скидку вычитают из полной стоимости, а не из количества участников.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
base = 80
percent = 5
final = base - base * percent / 100
print(f"{final:.2f}")  # 76.00
```

ОБРАТИ ВНИМАНИЕ

Вычитание 4 из суммы не означает скидку 4 процента.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси количество учеников >= 1, цену билета > 0 и скидку от 0 до 100 процентов. Выведи полную стоимость и сумму после скидки. Сделай имена переменных понятными.

ПРОВЕРЬ СЕБЯ

- 23, 7.5, 4 полная 172.50; после скидки 165.60.
- 10, 8, 0 обе суммы 80.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 09 / 9 ИЗ 15 В ТЕМЕ

Длительность фильма

Средний Создать / ex09.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для неотрицательного целого числа // выделяет полные группы, а % - остаток. Это удобно для минут, упаковок и мест в автобусе.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
pieces = 38
print(pieces // 12, pieces % 12)  # 3 2
```

ОБРАТИ ВНИМАНИЕ

2.25 часа и 2 часа 15 минут - разные формы записи.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси длительность фильма в целых минутах >= 0. Выведи «Фильм длится H ч M мин», где M от 0 до 59. Обычное деление / для часов не используй.

ПРОВЕРЬ СЕБЯ

- 135 Фильм длится 2 ч 15 мин.
- 59 0 ч 59 мин; 60 1 ч 0 мин.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 10 / 10 ИЗ 15 В ТЕМЕ

Время загрузки

Средний

Создать / ex10.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Перед делением приведи единицы к одной системе. В этой задаче 1 ГБ = 1000 МБ, 1 байт = 8 бит; это условленная десятичная модель.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
megabytes = 250
megabits = megabytes * 8
seconds = round(megabits / 40)
print(seconds)  # 50
```

ОБРАТИ ВНИМАНИЕ

Мегабайты и мегабиты не равны. round() при точной половине выбирает ближайшее чётное целое.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси размер файла в ГБ > 0 и скорость в Мбит/с > 0. Вычисли секунды, округли их round() до целого, затем выведи минуты и оставшиеся секунды.

ПРОВЕРЬ СЕБЯ

- 4.7 и 50 12 мин 32 с.
- 4.7 и 500 1 мин 15 с.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 11 / 11 ИЗ 15 В ТЕМЕ

Три цвета по кругу

Сложный

Создать / ex11.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Если значение понадобится позже, сохрани его до перезаписи. Временная переменная помогает не потерять данные при перестановке.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
left = "кот"
right = "пёс"
saved = left
left = right
right = saved
```

ОБРАТИ ВНИМАНИЕ

Запиши таблицу значений: каждый шаг должен сохранять ещё нужные цвета.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Дано: left = "красный", middle = "зелёный", right = "синий". Сдвинь цвета вправо: последний становится первым. Используй ровно одну дополнительную переменную; параллельное присваивание пока запрещено. Покажи строки «До» и «После».

ПРОВЕРЬ СЕБЯ

- После: синий красный зелёный.
- Повтори мысленно с A, B, C: должно стать C, A, B.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 12 / 12 ИЗ 15 В ТЕМЕ

Цифры числа

Сложный Создать / ex12.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для выделения разряда нужны целочисленное деление и остаток. Числовое представление не хранит нули перед первой значащей цифрой.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
n = 86
tens = n // 10
units = n % 10
print(tens, units)  # 8 6
```

ОБРАТИ ВНИМАНИЕ

Обратное число и текст «001» - разные результаты.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси трёхзначное целое n от 100 до 999. Только вычислениями выдели сотни, десятки и единицы. Выведи их сумму и число с обратным порядком цифр. str() и обработку текста не используй.

ПРОВЕРЬ СЕБЯ

- 472 цифры 4, 7, 2; сумма 13; обратное число 274.
- 905 509; 100 1. Объясни потерю двух нулей.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 13 / 13 ИЗ 15 В ТЕМЕ

Монеты для сдачи

Сложный Создать / ex13.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

При размене можно последовательно выделять полные крупные номиналы и продолжать с остатком. Для набора 50, 20, 10, 5, 2, 1 такой выбор даёт минимальное число единиц.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
amount = 17
coins_5 = amount // 5
rest = amount % 5
print(coins_5, rest)  # 3 2
```

ОБРАТИ ВНИМАНИЕ

Остаток после каждого номинала становится входом для следующего.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Сумма - целое количество евро >= 0. Размени её купюрами 50, 20, 10, 5 и монетами 2, 1. Без циклов выведи количество каждого номинала, включая нули. Восстанови сумму из результата для проверки.

ПРОВЕРЬ СЕБЯ

- 87 количества 1, 1, 1, 1, 1, 0.
- 9 0, 0, 0, 1, 2, 0; 0 все количества 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 14 / 14 ИЗ 15 В ТЕМЕ

Поезд через полночь

Сложный

Создать / ex14.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Сутки содержат 1440 минут. Остаток по модулю 1440 позволяет вернуть разницу времён в диапазон от 0 до 1439.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
position = 6
next_position = (position + 3) % 8
print(next_position)  # 1
```

ОБРАТИ ВНИМАНИЕ

Вычитать отдельно часы и минуты неудобно; сначала считай минуты от полуночи.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси часы и минуты отправления и прибытия: часы 0..23, минуты 0..59. Поезд едет меньше 24 часов; одинаковые времена означают нулевую длительность. Выведи часы и минуты пути. Условия и циклы запрещены.

ПРОВЕРЬ СЕБЯ

- 14:35 17:10: 2 ч 35 мин.
- 23:40 01:15: 1 ч 35 мин; 08:00 08:00: 0 ч 0 мин.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 15 / 15 ИЗ 15 В ТЕМЕ

Почасовая парковка

Сложный

Создать / ex15.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Начатый час оплачивается целиком. Для целых минут округление вверх удобно выразить через целочисленное деление с добавкой; при нуле результат должен быть нулевым.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
items = 13
box_size = 5
boxes = (items + box_size - 1) // box_size
print(boxes)  # 3
```

ОБРАТИ ВНИМАНИЕ

Ровно два часа нельзя округлять до трёх.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Автомобиль въехал и выехал в один день; выезд не раньше въезда. Запроси четыре значения времени. Цена - 2.50 евро за начатый час. Выведи длительность, число оплачиваемых часов и сумму. Нулевая длительность бесплатна.

ПРОВЕРЬ СЕБЯ

- 08:47 17:25: 8 ч 38 мин; 9 часов; 22.50 евро.
- 10:00 12:00: 5.00; 12:01: 7.50; 10:00: 0.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ТЕМА 2 / ЗАДАНИЯ 16-30

Условия: if, elif, else

Пусть программа выбирает действие по данным.

- Условие даёт True или False. Сравнения: ==, !=, <, <=, >, >=. После if и elif ставится двоеточие. Тело ветви записывается с отступом в четыре пробела.
- В цепочке if / elif / else выполняется первая подходящая ветвь. elif означает «иначе, если», else - все оставшиеся случаи. Несколько отдельных if проверяются независимо.
- and требует выполнения обоих условий, or - хотя бы одного, not меняет истину на ложь и наоборот. Скобки помогают сделать сложное условие понятным.
- Проверяй границы: значение ровно на пороге, на единицу меньше и на единицу больше. Сначала отсекай недопустимые данные, затем вычисляй результат.

ОСНОВНОЙ СИНТАКСИС

```
score = 12
if score >= 15:
    print("Высокий результат")
elif score >= 10:
    print("Зачёт")
else:
    print("Нужна практика")
```

ЧТО ПРИГОДИТСЯ

if условие:	Первая проверка
elif условие:	Следующая проверка, если предыдущие ложны
else:	Все оставшиеся случаи
a <= x <= b	Проверка включительного диапазона
and / or / not	Соединение и отрицание условий

ЧАСТЫЕ ОШИБКИ

Не путай = и ==. else не содержит условия. Не ставь самые широкие диапазоны раньше узких, если они перекрываются.

ЗАДАНИЕ 16 / 1 ИЗ 15 В ТЕМЕ

Знак числа

Начальный Создать / ex16.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для трёх взаимоисключающих случаев подходит одна цепочка if / elif / else. Ноль следует обработать отдельно от положительных чисел.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
temperature = 3
if temperature < 0:
    print("Мороз")
else:
    print("Не мороз")
```

ОБРАТИ ВНИМАНИЕ

Проверка >= 0 включает ноль.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целое число. Выведи ровно одно слово: «положительное», «отрицательное» или «ноль». Используй if, elif и else.

ПРОВЕРЬ СЕБЯ

- 7 положительное; -4 отрицательное.
- 0 ноль; результат должен занимать одну строку.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 17 / 2 ИЗ 15 В ТЕМЕ

Чётное или нечётное

Начальный

Создать / ex17.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Если число делится на 2 без остатка, оно чётное. Проверку делимости записывают n % divisor == 0.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
pages = 15
if pages % 3 == 0:
    print("Можно разделить поровну на троих")
```

ОБРАТИ ВНИМАНИЕ

Чётность определяют по остатку, а не по последней цифре, полученной вручную.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целое число и выведи «чётное» или «нечётное». Программа должна работать также с нулём и отрицательными числами.

ПРОВЕРЬ СЕБЯ

- 8 чётное; 7 нечётное.
- 0 и -6 чётное; -5 нечётное.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 18 / 3 ИЗ 15 В ТЕМЕ

Какая ветвь сработает?

Начальный

Прочитать / ex18.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

В одной цепочке после первой истинной проверки остальные ветви пропускаются. Порядок проверок влияет на результат.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
x = 7
if x > 10:
    print("A")
elif x > 5:
    print("B")
```

ОБРАТИ ВНИМАНИЕ

Условие score >= 10 тоже верно при 20, но может уже не проверяться.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Предскажи вывод кода для score = 9, 10, 14, 15 и 20. Для каждого значения назови первую истинную проверку. Затем объясни, почему две строки одновременно не появятся.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
score = 15
if score >= 15:
    print("Отлично")
elif score >= 10:
    print("Зачёт")
else:
    print("Повторить")
```

ПРОВЕРЬ СЕБЯ

- Составь пять строк таблицы: score, проверка, вывод.
- После прогноза запусти код с каждым значением.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 19 / 4 ИЗ 15 В ТЕМЕ

Кино по возрасту

Начальный

Создать / ex19.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Пороговое правило нужно записать с правильным знаком. >= включает сам порог, > его исключает.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
height = 140
if height >= 130:
    print("Рост подходит")
else:
    print("Рост пока мал")
```

ОБРАТИ ВНИМАНИЕ

Тест на 12 обязателен: именно он проверяет границу.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

В учебной модели билет доступен с 12 лет включительно. Запроси целый возраст >= 0. Если возраст меньше 12, выведи «Пока рано», иначе «Можно купить билет». Это правило задачи, а не реальная возрастная норма.

ПРОВЕРЬ СЕБЯ

- 11 Пока рано; 12 Можно купить билет.
- 0 Пока рано; 18 Можно купить билет.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 20 / 5 ИЗ 15 В ТЕМЕ

Пропущено двоеточие

Начальный Исправить / ex20.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Двоеточие начинает блок. Отступ показывает, какие инструкции принадлежат условию. SyntaxError связан с записью, IndentationError - с отступами.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
ready = True
if ready:
    print("Старт")
```

ОБРАТИ ВНИМАНИЕ

Исправляй ошибки по одной и запускай файл снова.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь две ошибки записи в коде. После исправления он должен выводить «Зачёт» при score >= 10 и «Незачёт» в остальных случаях. Запиши сообщения ошибок, которые увидел.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
score = 12
if score >= 10
print("Зачёт")
else:
    print("Незачёт")
```

ПРОВЕРЬ СЕБЯ

- score = 12 Зачёт.
- score = 9 Незачёт; score = 10 Зачёт.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 21 / 6 ИЗ 15 В ТЕМЕ

Шкала оценок

Средний

Создать / ex21.py

Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Если диапазоны идут подряд, проверяй верхние пороги первыми. Сначала выдели неверные значения, чтобы они не получили обычную категорию.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
speed = 70
if speed > 90:
    print("Высоко")
elif speed >= 50:
    print("Средне")
else:
    print("Низко")
```

ОБРАТИ ВНИМАНИЕ

Каждая допустимая оценка должна попасть ровно в один диапазон.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целую оценку. Вне 0..20 выведи «Ошибка». В пределах шкалы: 16..20 - «Отлично», 12..15 - «Хорошо», 10..11 - «Зачёт», 0..9 - «Незачёт». Вывод - одна категория.

ПРОВЕРЬ СЕБЯ

- 16 Отлично; 15 Хорошо; 10 Зачёт; 9 Незачёт.
- -1 и 21 Ошибка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 22 / 7 ИЗ 15 В ТЕМЕ

Цена доставки

Средний

Создать / ex22.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

and соединяет обязательные условия. or удобно использовать, когда хотя бы один признак делает данные неверными.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
weight = 2
fragile = "да"
if weight < 3 and fragile == "нет":
    print("Обычная упаковка")
```

ОБРАТИ ВНИМАНИЕ

Бесплатный порог относится только к центру.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси стоимость заказа >= 0 и город: «центр» или «другой». В центре доставка бесплатна от 50 евро, иначе 4 евро. В другом городе она всегда 7 евро. Для неизвестного города выведи «Ошибка города». Покажи цену доставки с двумя знаками.

ПРОВЕРЬ СЕБЯ

- центр, 49.99 4.00; центр, 50 0.00.
- другой, 100 7.00; север, 60 Ошибка города.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 23 / 8 ИЗ 15 В ТЕМЕ

Наибольшее из трёх

Средний

Создать / ex23.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Сравнивай кандидата с остальными значениями или последовательно обновляй текущий максимум. Равенство тоже должно давать результат.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = 8
b = 3
if a >= b:
    biggest = a
else:
    biggest = b
```

ОБРАТИ ВНИМАНИЕ

Начальное значение 0 не подходит, если все входные числа отрицательные.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси три целых числа. Выведи наибольшее. Используй условия; встроенная max() запрещена. Программа должна работать при равных и отрицательных значениях.

ПРОВЕРЬ СЕБЯ

- 4, 9, 2 9; 7, 7, 1 7.
- -5, -2, -8 -2; 3, 3, 3 3.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 24 / 9 ИЗ 15 В ТЕМЕ

Двойное сообщение

Средний

Исправить / ex24.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Несколько if - независимые решения. Если нужен один результат, связанные проверки стоит объединить в цепочку или изменить их диапазоны.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
mode = 2
if mode == 1:
    print("Тихо")
elif mode == 2:
    print("Обычно")
```

ОБРАТИ ВНИМАНИЕ

else относится к ближайшему if на том же уровне отступа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Сейчас при сумме 120 выводятся две скидки. Исправь код: от 100 евро скидка 10%, от 50 до 99.99 - 5%, меньше 50 - 0%. Выведи только одну строку. Добавь комментарий о причине ошибки.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
total = 120
if total >= 100:
    print("Скидка 10%")
if total >= 50:
    print("Скидка 5%")
else:
    print("Без скидки")
```

ПРОВЕРЬ СЕБЯ

- 120 Скидка 10%; 50 Скидка 5%.
- 49.99 Без скидки; 100 Скидка 10%.

ЗАДАНИЕ 25 / 10 ИЗ 15 В ТЕМЕ

Калькулятор с защитой

Средний

Создать / ex25.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Проверка должна происходить до опасного действия. Если делитель нулевой, обычное деление вызовет ZeroDivisionError.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
count = 0
if count == 0:
    print("Нет элементов")
else:
    print(30 / count)
```

ОБРАТИ ВНИМАНИЕ

Проверить ноль после вычисления уже поздно.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси два числа и операцию из +, -, *, /. Выведи результат с двумя знаками. Для деления на ноль выведи «Деление на ноль», для иной операции - «Неизвестная операция».

ПРОВЕРЬ СЕБЯ

- 8, 2, / 4.00; 3, 4, * 12.00.
- 5, 0, / Деление на ноль; 2, 3, ? Неизвестная операция.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 26 / 11 ИЗ 15 В ТЕМЕ

Високосный год

Сложный Создать / ex26.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

В сложном правиле выделяй исключения. Скобки позволяют явно показать, как and и or собирают одно логическое выражение.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
n = 18
suitable = (n % 2 == 0) and (n % 3 == 0)
print(suitable)  # True
```

ОБРАТИ ВНИМАНИЕ

Правило «делится на 4» не учитывает исключение для столетий.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Год >= 1 високосный, если делится на 4, кроме кратных 100; кратные 400 всё же високосные. Запроси год, составь одно логическое выражение и выведи «Високосный» или «Обычный».

ПРОВЕРЬ СЕБЯ

- 2024 и 2000 Високосный.
- 2026 и 1900 Обычный.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 27 / 12 ИЗ 15 В ТЕМЕ

Существует ли треугольник?

Сложный Создать / ex27.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для треугольника сумма любых двух сторон строго больше третьей. Положительность сторон - отдельное обязательное требование.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
x = 4
allowed = x > 0 and x < 10
print(allowed)  # True
```

ОБРАТИ ВНИМАНИЕ

Равенство суммы двух сторон третьей даёт отрезок, а не треугольник.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси три целые длины. Если хотя бы одна <= 0 или не выполнены неравенства треугольника, выведи «Не существует». Иначе классифицируй: «Равносторонний», «Равнобедренный» или «Разносторонний».

ПРОВЕРЬ СЕБЯ

- 3, 3, 3 Равносторонний; 3, 3, 5 Равнобедренный.
- 3, 4, 5 Разносторонний; 1, 2, 3 и 0, 2, 2 Не существует.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 28 / 13 ИЗ 15 В ТЕМЕ

Скидка и промокод

Сложный

Создать / ex28.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Если несколько правил влияют на результат, сначала выбери процент, затем применяй его к исходной сумме один раз.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
base = 200
percent = 10
final = base * (1 - percent / 100)
print(f"{final:.2f}")  # 180.00
```

ОБРАТИ ВНИМАНИЕ

Добавить 3 процентных пункта - не то же самое, что уменьшить уже сниженный итог ещё на 3%.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси сумму >= 0 и код. От 100 евро базовая скидка 10%, от 50 - 5%, иначе 0%. Точный код STUDENT добавляет 3 процентных пункта. Общая скидка не выше 12%. Выведи процент и итоговую сумму.

ПРОВЕРЬ СЕБЯ

- 120, STUDENT 12%; 105.60 евро.
- 60, STUDENT 8%; 55.20; 40, student 0%; 40.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 29 / 14 ИЗ 15 В ТЕМЕ

Билет на поезд

Сложный

Создать / ex29.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Вложенные условия полезны, когда второй выбор зависит от первого. До вычислений проверь недопустимые значения.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
age = 9
if age < 18:
    rate = 0.5
else:
    rate = 1.0
```

ОБРАТИ ВНИМАНИЕ

Бесплатный билет должен остаться бесплатным даже в выходной.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Правила учебного перевозчика: базовый билет 20 евро; до 6 лет - бесплатно, 6..17 - половина, 18..64 - полная цена, от 65 - 70% цены. Если день «выходной», к полученной цене добавь 2 евро, кроме бесплатного билета. Дни: «будний»/«выходной»; неверный день или возраст < 0 «Ошибка».

ПРОВЕРЬ СЕБЯ

- 5, выходной 0.00; 6, выходной 12.00.
- 65, будний 14.00; -1, будний Ошибка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 30 / 15 ИЗ 15 В ТЕМЕ

Тариф за воду

Сложный

Создать / ex30.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Ступенчатый тариф считают по частям: новая цена относится только к объёму сверх порога. Общий объём нельзя целиком умножить на последнюю ставку.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
units = 7
if units <= 5:
    cost = units * 2
else:
    cost = 5 * 2 + (units - 5) * 3
```

ОБРАТИ ВНИМАНИЕ

При 25 м³ ставка 3.00 применяется только к последним пяти кубометрам.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целый расход воды в м³ >= 0. Первые 10 м³ стоят 1.50 евро каждый, следующие 10 - 2.00, весь объём сверх 20 - 3.00. Выведи счёт с двумя знаками. Отрицательный ввод «Ошибка».

ПРОВЕРЬ СЕБЯ

- 0 0.00; 10 15.00; 11 17.00.
- 20 35.00; 25 50.00; -1 Ошибка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ТЕМА 3 / ЗАДАНИЯ 31-45

Циклы: for и while

Повторяй действия без копирования строк кода.

- Одна итерация - один проход цикла. for удобен, когда известен набор значений: for i in range(...). range(start, stop, step) не включает stop; шаг не может быть нулевым.
- while повторяет блок, пока условие истинно. Проверь начальное состояние, изменение данных и момент остановки. Если условие никогда не станет ложным, получится бесконечный цикл.
- Счётчик хранит количество, накопитель - сумму или другой общий результат. Создавай их до цикла; обновляй внутри. Запись total += x означает total = total + x.
- break завершает ближайший цикл, continue переходит к следующей итерации. Во вложенных циклах внутренний полностью выполняется для каждого значения внешнего.

ОСНОВНОЙ СИНТАКСИС

```
total = 0
for number in range(1, 5):
    total += number
print(total)  # 10

count = 3
while count > 0:
    print(count)
    count -= 1
```

ЧТО ПРИГОДИТСЯ

range(5)	Значения 0, 1, 2, 3, 4
range(2, 7)	Значения от 2 до 6
range(6, 0, -2)	Значения 6, 4, 2
x += 1; x -= 1	Увеличить или уменьшить на 1
break / continue	Выйти из цикла / пропустить остаток прохода

ЧАСТЫЕ ОШИБКИ

Не забывай менять данные в while. Не обнуляй накопитель на каждом проходе. Если нужна включительная верхняя граница n, обычно stop равен n + 1.

ЗАДАНИЕ 31 / 1 ИЗ 15 В ТЕМЕ

Пять приветствий

Начальный

Создать / ex31.py

Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

for выполняет тело по одному разу на каждое значение range(). Если само значение не нужно, имя переменной всё равно должно быть указано.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for i in range(3):
    print("Практика")
```

ОБРАТИ ВНИМАНИЕ

range(5) начинается с 0, но всё равно содержит пять значений.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Выведи «Привет, Python!» ровно пять раз, каждое приветствие с новой строки. В коде должен быть один print() внутри одного for.

ПРОВЕРЬ СЕБЯ

- Посчитай строки: их ровно пять.
- Замени количество повторений на 1: должна остаться одна строка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 32 / 2 ИЗ 15 В ТЕМЕ

Последнее число потерялось

Начальный

Исправить / ex32.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Правая граница range() не входит в последовательность. Ошибка на единицу часто оставляет незамеченным последний элемент.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for n in range(2, 5):
    print(n)  # 2, затем 3, затем 4
```

ОБРАТИ ВНИМАНИЕ

Тест n = 1 особенно хорошо показывает неверную границу.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Программа должна выводить все целые от 1 до введённого n включительно; n >= 1. Исправь диапазон. Объясни в комментарии, какое значение терялось.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
n = int(input("n: "))
for i in range(1, n):
    print(i)
```

ПРОВЕРЬ СЕБЯ

- n = 4 отдельные строки 1, 2, 3, 4.
- n = 1 одна строка 1.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 33 / 3 ИЗ 15 В ТЕМЕ

Следим за суммой

Начальный

Прочитать / ex33.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Накопитель сохраняет результат предыдущих проходов. Таблица с колонками «значение цикла» и «сумма после прохода» помогает увидеть вычисление.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
total = 0
for x in range(2, 4):
    total += x
print(total)  # 5
```

ОБРАТИ ВНИМАНИЕ

Инициализация и обновление накопителя - разные действия.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Без запуска выпиши все значения i и total после каждого прохода. Затем предскажи вывод. Объясни, как изменится результат, если total = 0 перенести внутрь цикла перед total += i.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
total = 0
for i in range(1, 5):
    total += i
print(total)
```

ПРОВЕРЬ СЕБЯ

- В таблице должно быть четыре прохода.
- Проверь прогноз запуском двух вариантов.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 34 / 4 ИЗ 15 В ТЕМЕ

Таблица умножения

Начальный

Создать / ex34.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Переменная цикла может участвовать в вычислениях и в тексте. f-строка позволяет выводить и исходные числа, и результат.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for n in range(1, 4):
    print(f"{n} + 2 = {n + 2}")
```

ОБРАТИ ВНИМАНИЕ

Не пиши десять отдельных print(): вычисления должны быть в цикле.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целое число от 1 до 12. Выведи для него таблицу умножения на числа от 1 до 10 включительно. Формат строки: «7 x 3 = 21».

ПРОВЕРЬ СЕБЯ

- Для 7 первая строка 7 x 1 = 7, последняя 7 x 10 = 70.
- Для 1 результаты идут от 1 до 10.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 35 / 5 ИЗ 15 В ТЕМЕ

Обратный отсчёт завис

Начальный Исправить / ex35.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Условие while перепроверяется перед каждым проходом. Чтобы выйти из цикла, нужно изменять хотя бы одно значение, от которого зависит условие.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
left = 2
while left > 0:
    print(left)
    left -= 1
```

ОБРАТИ ВНИМАНИЕ

Обновление вне тела while не поможет завершить этот цикл.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь цикл, который должен вывести 5, 4, 3, 2, 1 и после этого «Старт». Сначала найди ошибку глазами: исходный вариант бесконечен. Если случайно запустил его, останови Ctrl+C.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
count = 5
while count > 0:
    print(count)
print("Старт")
```

ПРОВЕРЬ СЕБЯ

- После исправления числа идут по убыванию.
- При count = 0 должны сразу появиться только слова «Старт».

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 36 / 6 ИЗ 15 В ТЕМЕ

Сумма до сигнала

Средний Создать / ex36.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Сигнальное значение завершает ввод и не считается обычными данными. Можно читать новое число в while True и выходить через break.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
while True:
    word = input("Слово или стоп: ")
    if word == "стоп":
        break
```

ОБРАТИ ВНИМАНИЕ

После сигнала не запрашивай ещё одно число.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Вводи целые числа по одному, пока не введён 0. Выведи сумму всех чисел до нуля и их количество. Отрицательные числа разрешены; 0 в количество не входит.

ПРОВЕРЬ СЕБЯ

- 5, -2, 7, 0 сумма 10; количество 3.
- Первый ввод 0 сумма 0; количество 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 37 / 7 ИЗ 15 В ТЕМЕ

Среднее за неделю

Средний

Создать / ex37.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Когда количество входных данных известно, используй for. Среднее считают после завершения накопления всей суммы.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
total = 0
for i in range(3):
    total += int(input("Очки: "))
print(total / 3)
```

ОБРАТИ ВНИМАНИЕ

Дели на n один раз после цикла, а не дели каждую температуру отдельно без необходимости.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси количество дней n >= 1. Затем введи n дробных температур, по одной на строку. Выведи среднюю температуру с одним знаком после точки. Списки пока не используй.

ПРОВЕРЬ СЕБЯ

- 3; затем 10, 12, 14 12.0.
- 1; затем -5 -5.0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 38 / 8 ИЗ 15 В ТЕМЕ

Сумма цифр любого числа

Средний

Создать / ex38.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Остаток n % 10 даёт последнюю цифру неотрицательного числа. После n //= 10 эта цифра удаляется; повторяй, пока цифры не закончатся.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
n = 64
while n > 0:
    print(n % 10)
    n //= 10
```

ОБРАТИ ВНИМАНИЕ

Введённое число можно сохранить отдельно, если хочешь показать его в итоговой фразе.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целое n >= 0. С помощью while найди сумму его цифр. Не преобразовывай число в строку. При вводе 0 сумма должна быть 0.

ПРОВЕРЬ СЕБЯ

- 472 13; 905 14.
- 0 0; 100000 1.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 39 / 9 ИЗ 15 В ТЕМЕ

Три попытки пароля

Средний

Создать / ex39.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Счётчик попыток ограничивает повторение. После успеха break должен остановить дальнейшие запросы.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for attempt in range(1, 3):
    answer = input("Код: ")
    if answer == "ok":
        print("Готово")
        break
```

ОБРАТИ ВНИМАНИЕ

Сообщение о неудаче не должно появляться после успешного входа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Учебный пароль - точная строка python. Дай максимум три попытки. При успехе выведи «Доступ открыт» и остановись. Если все три неверны, выведи «Попытки закончились». Сравнение чувствительно к регистру.

ПРОВЕРЬ СЕБЯ

- python с первого раза один запрос и Доступ открыт.
- a, b, python успех с третьего; a, b, c Попытки закончились.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 40 / 10 ИЗ 15 В ТЕМЕ

Пропускаем кратные трём

Средний

Создать / ex40.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

continue пропускает остаток текущего прохода. Проверка для пропуска должна стоять до действия, которое нужно пропустить.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for n in range(1, 6):
    if n == 3:
        continue
    print(n)
```

ОБРАТИ ВНИМАНИЕ

Накопление до continue ошибочно включит пропущенные числа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси n >= 1. Перебери числа от 1 до n включительно и с помощью continue пропусти кратные 3. Выведи каждое оставшееся число на отдельной строке, затем их сумму.

ПРОВЕРЬ СЕБЯ

- n = 7 числа 1, 2, 4, 5, 7; сумма 19.
- n = 3 числа 1, 2; сумма 3.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 41 / 11 ИЗ 15 В ТЕМЕ

Простое ли число?

Сложный Создать / ex41.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Простое число - целое больше 1, у которого только два положительных делителя: 1 и оно само. Достаточно найти один иной делитель, чтобы опровергнуть простоту.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
n = 12
for divisor in range(2, n):
    if n % divisor == 0:
        print("Нашёл делитель", divisor)
        break
```

ОБРАТИ ВНИМАНИЕ

Отсутствие проходов цикла для 1 не делает число простым.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси целое n >= 0. Проверь простоту перебором делителей и выведи «Простое» или «Не простое». При найденном делителе прекращай поиск. Для 0 и 1 сразу нужен ответ «Не простое».

ПРОВЕРЬ СЕБЯ

- 2 и 17 Простое.
- 0, 1, 9 и 21 Не простое.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 42 / 12 ИЗ 15 В ТЕМЕ

Лестница из символов

Сложный

Создать / ex42.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Вложенный цикл строит одну строку по частям. print(..., end="") не переводит строку; отдельный print() завершает её.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
for row in range(2):
    for col in range(3):
        print("+", end="")
    print()
```

ОБРАТИ ВНИМАНИЕ

Перевод строки должен происходить после внутреннего цикла, а не после каждой звезды.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси высоту h от 1 до 10. В строке номер k выведи k звёздочек: первая содержит одну, последняя h. Используй два вложенных for; умножение строки на число запрещено.

ПРОВЕРЬ СЕБЯ

- h = 3 строки *, **, ***.
- h = 1 только одна звёздочка и одна строка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 43 / 13 ИЗ 15 В ТЕМЕ

Копилка до цели

Сложный

Создать / ex43.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

while подходит, когда число шагов неизвестно заранее. Состояние меняется на каждом шаге, а условие проверяет достижение цели.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
stored = 0
steps = 0
while stored < 12:
    stored += 5
    steps += 1
print(steps)  # 3
```

ОБРАТИ ВНИМАНИЕ

Взнос 0 не допускается условием: иначе недостижимая цель дала бы бесконечный цикл.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси текущие целые накопления >= 0, целевую сумму >= 0 и еженедельный взнос > 0. Циклом найди число недель до достижения или превышения цели. Если цель уже достигнута, нужно 0 недель. Покажи итоговые накопления.

ПРОВЕРЬ СЕБЯ

- 20, цель 100, взнос 15 6 недель; итог 110.
- 100, цель 100, взнос 10 0 недель; итог 100.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 44 / 14 ИЗ 15 В ТЕМЕ

Наибольший общий делитель

Сложный

Создать / ex44.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Алгоритм Евклида заменяет пару чисел вторым числом и остатком от деления первого на второе. Когда второе равно нулю, первое - НОД.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = 18
b = 12
remainder = a % b
print(remainder)  # 6
```

ОБРАТИ ВНИМАНИЕ

Остаток нужно вычислить до перезаписи первого числа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси два положительных целых. Найди НОД с помощью while и остатков, без math.gcd(). Для первых входных данных запиши в комментарии все пары чисел по шагам. Можно использовать временную переменную.

ПРОВЕРЬ СЕБЯ

- 48 и 18 6; 17 и 13 1.
- 12 и 12 12; 1 и 25 1.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 45 / 15 ИЗ 15 В ТЕМЕ

Мини-кошелёк с меню

Сложный

Создать / ex45.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Цикл меню читает команду, выполняет действие и возвращается к следующему запросу. Выход - отдельная команда; состояние хранится между проходами.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
while True:
    command = input("Команда: ")
    if command == "0":
        break
    print("Получено:", command)
```

ОБРАТИ ВНИМАНИЕ

При отказе баланс не меняется. Не запрашивай сумму для команды 3 или 0.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Начальный баланс 0 целых евро. Команды: 1 - пополнить, 2 - списать, 3 - показать баланс, 0 - выйти. Для 1 и 2 запроси положительную целую сумму. Неположительная сумма «Ошибка суммы», списание сверх баланса «Недостаточно средств», иная команда «Неизвестная команда».

ПРОВЕРЬ СЕБЯ

- 1, 10, 2, 3, 3, 0 после операций показан баланс 7.
- 2, 1 отказ; 1, 0 ошибка суммы; баланс остаётся 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ТЕМА 4 / ЗАДАНИЯ 46-60

Строки и обработка текста

Исправляй ввод, ищи символы и преобразуй слова.

- Строка - последовательность символов. len(text) даёт длину, text[0] - первый символ, text[-1] - последний. У пустой строки нет первого символа.
- Срез text[start:stop:step] берёт часть строки; stop не включается. text[:3] - первые три символа, text[-4:] - последние четыре, text[::-1] - обратный порядок.
- Строки неизменяемы. strip(), lower(), upper(), capitalize() и replace() возвращают новую строку. Чтобы сохранить результат, присвой его переменной.
- in проверяет наличие фрагмента. find() даёт индекс первого совпадения или -1. split() без аргумента разбивает текст по пробельным символам и возвращает список слов - упорядоченный набор строк.

ОСНОВНОЙ СИНТАКСИС

```
text = "  КОТ  "
clean = text.strip().lower()
print(clean)        # кот
print(clean[0])     # к
print(clean[::-1])  # ток
```

ЧТО ПРИГОДИТСЯ

text.strip()	Убрать пробельные символы по краям
text.lower() / upper()	Изменить регистр
text.count(part)	Посчитать неперекрывающиеся вхождения
text.find(part)	Найти позицию; -1 означает «не найдено»
text.replace(a, b)	Заменить все вхождения a на b

ЧАСТЫЕ ОШИБКИ

Не обращайся к text[0] без проверки пустого ввода. find() может вернуть -1. Простая проверка e-mail или пароля в задаче не является полной проверкой реального адреса или безопасности.

ЗАДАНИЕ 46 / 1 ИЗ 15 В ТЕМЕ

Первый и последний символ

Начальный

Создать / ex46.py

Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Индексы начинаются с 0. Отрицательный индекс -1 обращается к последнему символу. Перед индексированием пустую строку нужно обработать отдельно.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
word = "дом"
print(len(word))  # 3
print(word[0])    # д
```

ОБРАТИ ВНИМАНИЕ

Не запускай индексирование до проверки длины.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси строку без удаления пробелов. Выведи её длину, первый и последний символ. Для пустой строки выведи только «Пустая строка». Пробел считается символом.

ПРОВЕРЬ СЕБЯ

- Python длина 6; первый P; последний n.
- Один символ Я первый и последний Я; пустой ввод Пустая строка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 47 / 2 ИЗ 15 В ТЕМЕ

Имя в аккуратном виде

Начальный

Создать / ex47.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

strip() очищает края, lower() уменьшает регистр, capitalize() делает первый символ заглавным, а остальные строчными.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
city = "  льЕЖ  "
print(city.strip().capitalize())  # Льеж
```

ОБРАТИ ВНИМАНИЕ

В этой задаче не рассматриваются составные фамилии и особые правила написания имён.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси имя и фамилию: каждое значение - одно слово, допускаются пробелы по краям. Выведи «Имя Фамилия» с заглавными первыми буквами. Входные строки не перепечатывай вручную.

ПРОВЕРЬ СЕБЯ

- тОМ и лАМБЕРТ Том Ламберт.
- анНА и ИВАНОВА Анна Иванова.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 48 / 3 ИЗ 15 В ТЕМЕ

Замена ничего не поменяла

Начальный Исправить / ex48.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Методы строк не изменяют исходный объект. Если возвращённая строка проигнорирована, переменная сохранит старый текст.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
1 line = "Встреча в 9:00"
2 line = line.replace("9:00", "9:15")
3 print(line)
```

ОБРАТИ ВНИМАНИЕ

replace() возвращает строку; результат нужно использовать.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Скрипт должен перенести начало урока с 8:30 на 8:20. Исправь код без переписывания всей фразы вручную. Объясни, почему старый вариант не сохранял замену.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
1 phrase = "Урок начинается в 8:30."
2 phrase.replace("8:30", "8:20")
3 print(phrase)
```

ПРОВЕРЬ СЕБЯ

- После исправления: Урок начинается в 8:20.
- Во фразе «8:30 и снова 8:30» должны замениться оба времени.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 49 / 4 ИЗ 15 В ТЕМЕ

Маскируем номер

Начальный

Создать / ex49.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Срез с отрицательной границей берёт символы с конца. Строку можно повторить: "*" * 3 даст три звёздочки.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
code = "ABCDEF"
print("#" * 4 + code[-2:])  # ####EF
```

ОБРАТИ ВНИМАНИЕ

Звёздочки не должны быть фиксированной длины.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси номер из цифр длиной не меньше четырёх. Скрой все символы, кроме последних четырёх, звёздочками. Количество звёздочек вычисляй по длине; вывод: «Номер: ...».

ПРОВЕРЬ СЕБЯ

- 4532015112830366 **********0366.
- 123456789 *****6789; 1234 1234.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 50 / 5 ИЗ 15 В ТЕМЕ

Что берёт срез?

Начальный

Прочитать / ex50.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Начало среза включается, конец исключается. Срез может быть пустым. В отличие от отдельного индекса, выход границы среза за длину не обязательно вызывает ошибку.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
text = "abcdef"
print(text[1:4])  # bcd
print(text[:2])   # ab
```

ОБРАТИ ВНИМАНИЕ

Символ по индексу stop в результат не входит.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Не запуская код, запиши результат каждой строки print(). Для каждого среза перечисли выбранные индексы. Затем проверь прогноз.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
word = "PYTHON"
print(word[:2])
print(word[2:5])
print(word[-2:])
print(word[::-1])
print(word[3:3])
```

ПРОВЕРЬ СЕБЯ

- Ответов должно быть пять, включая пустую строку, если она получается.
- Отдельно объясни действие шага -1.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 51 / 6 ИЗ 15 В ТЕМЕ

Палиндром без регистра

Средний Создать / ex51.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Перед сравнением приведи обе стороны к одной форме. Если правило разрешает игнорировать пробелы, убери именно их; остальные символы сохраняются.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
word = "ДоМ"
clean = word.lower()
print(clean == "дом")  # True
```

ОБРАТИ ВНИМАНИЕ

Сравнить перевёрнутую исходную строку с очищенной строкой недостаточно: обе формы должны совпадать.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси непустую фразу из букв и обычных пробелов. Игнорируя регистр и пробелы, проверь, одинаково ли она читается в обе стороны. Выведи «Палиндром» или «Не палиндром».

ПРОВЕРЬ СЕБЯ

- Radar и «А роза упала на лапу Азора» Палиндром.
- Robot Не палиндром.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 52 / 7 ИЗ 15 В ТЕМЕ

Сколько гласных?

Средний

Создать / ex52.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Цикл for может идти прямо по символам строки. Проверка symbol in alphabet удобна для принадлежности небольшому набору букв.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
total = 0
for ch in "банан":
    if ch == "а":
        total += 1
print(total)  # 2
```

ОБРАТИ ВНИМАНИЕ

Буква ё входит в набор и не должна потеряться.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Посчитай русские гласные в введённой строке: а, е, ё, и, о, у, ы, э, ю, я. Регистр не учитывай. Пробелы, знаки препинания и латинские буквы не считай.

ПРОВЕРЬ СЕБЯ

- «Привет, мир!» → 3; «Ёж и я» → 3.
- Python и пустая строка → 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 53 / 8 ИЗ 15 В ТЕМЕ

Разбираем учебный e-mail

Средний

Создать / ex53.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

find("@") показывает позицию разделителя. Срез слева заканчивается перед ним, справа начинается после него. Сначала проверь, есть ли разделитель и сколько их.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
record = "имя:значение"
pos = record.find(":")
print(record[:pos])  # имя
```

ОБРАТИ ВНИМАНИЕ

Это проверка по правилам задачи, а не полная проверка формата интернет-почты.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси строку. Допустимый учебный адрес содержит ровно один @, непустые части слева и справа и не содержит обычных пробелов. Для него выведи пользователя и домен. Иначе выведи «Некорректный адрес». Используй find() и срезы.

ПРОВЕРЬ СЕБЯ

- tom.lambert@ecole.be tom.lambert и ecole.be.
- a@@b, @site, name@, a b@c Некорректный адрес.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 54 / 9 ИЗ 15 В ТЕМЕ

Расширение файла

Средний

Создать / ex54.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

rfind() ищет последнее вхождение. lower() помогает сравнить расширения без учёта регистра. Точка в начале имени не обязательно задаёт расширение.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
name = "archive.old.txt"
pos = name.rfind(".")
print(name[pos + 1:])  # txt
```

ОБРАТИ ВНИМАНИЕ

Последняя точка важнее первой; отдельно проверь pos == -1.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси непустое имя без пути. Расширение - часть после последней точки, если точка не первая и не последняя. Выведи расширение строчными буквами, иначе «Без расширения».

ПРОВЕРЬ СЕБЯ

- photo.JPG jpg; archive.tar.gz gz.
- README, .env, report. Без расширения.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 55 / 10 ИЗ 15 В ТЕМЕ

Проверяем целую запись

Средний

Создать / ex55.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Проверка символов может предшествовать int(). В этом упражнении цифрами считаются только ASCII-символы 0..9, а не все символы, для которых isdigit() истинно.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
ch = "7"
print(ch in "0123456789")  # True
```

ОБРАТИ ВНИМАНИЕ

Один минус не содержит цифр; вызови int() только после проверки всей строки.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси строку и убери пробелы по краям. Разрешены необязательный начальный минус и хотя бы одна цифра 0..9. Проверяй символы циклом. При успехе выведи удвоенное целое число, иначе «Не целое». Плюс и дробная точка запрещены.

ПРОВЕРЬ СЕБЯ

- « -12 » -24; «007» 14.
- «-», «», «3.5», «+2», «1 2» Не целое.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 56 / 11 ИЗ 15 В ТЕМЕ

Шифр со сдвигом

Сложный

Создать / ex56.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Индекс буквы можно сдвинуть и вернуть в диапазон через %. Для алфавита из 26 символов индекс после сдвига берётся по модулю 26.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
alphabet = "ABC"
index = alphabet.find("C")
print(alphabet[(index + 1) % 3])  # A
```

ОБРАТИ ВНИМАНИЕ

Не ищи индекс пробела в алфавите: он обрабатывается отдельно.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси текст из заглавных латинских букв и пробелов и целый сдвиг k >= 0. Используй алфавит ABCDEFGHIJKLMNOPQRSTUVWXYZ. Сдвинь каждую букву вправо на k позиций по кругу; пробелы сохрани. Выведи шифртекст.

ПРОВЕРЬ СЕБЯ

- XYZ, k = 3 ABC; HELLO, k = 1 IFMMP.
- A Z, k = 26 A Z; пустой текст остаётся пустым.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 57 / 12 ИЗ 15 В ТЕМЕ

Группы одинаковых символов

Сложный

Создать / ex57.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Серия - соседние одинаковые символы. Когда символ меняется, заверши прежнюю серию. После цикла не забудь вывести последнюю.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
text = "aab"
print(text[0] == text[1])  # True
print(text[1] == text[2])  # False
```

ОБРАТИ ВНИМАНИЕ

В aba две отдельные серии a, хотя буква встречается дважды.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси строку только из строчных латинских букв; пустой ввод разрешён. Выведи каждую серию в формате буква:количество, серии раздели пробелом. Для пустого ввода выведи «Пусто». Общий подсчёт букв по всей строке не подходит.

ПРОВЕРЬ СЕБЯ

- aaabbc a:3 b:2 c:1.
- aba a:1 b:1 a:1; z z:1; пустой ввод Пусто.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 58 / 13 ИЗ 15 В ТЕМЕ

Самое длинное слово

Сложный

Создать / ex58.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

split() без аргумента делит строку по пробельным символам и возвращает список слов. for может перебирать эти слова без обращения к индексам.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
words = "кот   большой".split()
for word in words:
    print(len(word))  # 3, затем 7
```

ОБРАТИ ВНИМАНИЕ

Не заменяй найденное слово при равной длине, иначе победит последнее.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси фразу из букв и пробелов. Найди самое длинное слово и его длину; при равенстве выбери первое. Для пустого ввода или одних пробелов выведи «Нет слов». Используй split() и цикл, без max().

ПРОВЕРЬ СЕБЯ

- «кот программирование дом» программирование, 16.
- «мир кот» мир, 3; « » Нет слов.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 59 / 14 ИЗ 15 В ТЕМЕ

Шаблон уведомления

Сложный

Создать / ex59.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

replace() может подставлять значения в текстовый шаблон. Повторяющиеся маркеры заменяются все. Число для текстовой вставки можно получить через str().

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
template = "Привет, {name}!"
text = template.replace("{name}", "Оля")
print(text)
```

ОБРАТИ ВНИМАНИЕ

Последовательные замены сохраняй; не возвращайся каждый раз к исходному шаблону.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Есть шаблон «{name}, урок {course} начнётся в {time}. До встречи, {name}!». Запроси имя, курс и время; убери пробелы по краям. Если хоть одно поле пустое, выведи «Не хватает данных». Иначе замени все маркеры. Во вводе не будет фигурных скобок.

ПРОВЕРЬ СЕБЯ

- Анна, Python, 8:30 оба {name} заменены на Анна.
- Пустой курс Не хватает данных; ни один маркер не должен попасть в готовый текст.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 60 / 15 ИЗ 15 В ТЕМЕ

Лишние пробелы внутри

Сложный

Создать / ex60.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Нормализацию можно сделать проходом по строке: добавлять обычный символ и добавлять пробел только тогда, когда предыдущий добавленный символ не был пробелом.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
result = ""
for ch in "кот":
    result += ch
print(result)  # кот
```

ОБРАТИ ВНИМАНИЕ

У пустого результата ещё нет последнего символа: проверяй это перед result[-1].

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси строку с буквами и обычными пробелами. Удали пробелы по краям и замени каждую внутреннюю серию пробелов одним. split() и join() запрещены: используй цикл и накопление строки. Для одних пробелов результат пустой.

ПРОВЕРЬ СЕБЯ

- « учу Python каждый день » «учу Python каждый день».
- «а б» «а б»; « » пустая строка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ТЕМА 5 / ЗАДАНИЯ 61-75

Списки и наборы данных

Работай сразу с несколькими значениями.

- Список хранит упорядоченные элементы: marks = [12, 16, 9]. Индексы начинаются с 0. len(marks) - количество элементов, marks[-1] - последний.
- Списки изменяемы: items[0] = value заменяет элемент, append(value) добавляет в конец, pop() удаляет и возвращает последний. for item in items перебирает значения.
- sum(), min() и max() работают с числовыми данными; min() и max() без значения по умолчанию не подходят пустому списку. sorted(items) возвращает новый отсортированный список; items.sort() меняет исходный и возвращает None.
- Присваивание b = a не копирует список: оба имени ведут к одному объекту. Для отдельного списка используй a.copy() или a[:]. Здесь достаточно поверхностной копии, потому что элементы - числа или строки.

ОСНОВНОЙ СИНТАКСИС

```
scores = [8, 12, 15]
scores.append(10)
total = 0
for score in scores:
    total += score
print(total)  # 45
print(sorted(scores))  # [8, 10, 12, 15]
```

ЧТО ПРИГОДИТСЯ

items.append(x)	Добавить элемент в конец
items[i] = x	Изменить элемент с индексом i
x in items	Проверить наличие значения
items.copy()	Сделать отдельную поверхностную копию
items.count(x)	Посчитать число вхождений x

ЧАСТЫЕ ОШИБКИ

Не изменяй длину списка во время перебора того же списка без ясного плана. Не путай индекс с самим значением. Для пустого списка сначала выбери отдельный сценарий.

ЗАДАНИЕ 61 / 1 ИЗ 15 В ТЕМЕ

Расписание в списке

Начальный

Создать / ex61.py

Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Чтобы получить элемент, указывай его индекс в квадратных скобках. При трёх элементах допустимы индексы 0, 1 и 2.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
colors = ["белый", "жёлтый", "чёрный"]
print(colors[1])  # жёлтый
```

ОБРАТИ ВНИМАНИЕ

Индекс 3 для исходного списка находится за его пределами.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

В готовом файле дан список ["Математика", "Английский", "Python"]. Выведи первый и последний предмет, затем количество предметов. Используй индексы и len(), не вписывай названия повторно в print().

ПРОВЕРЬ СЕБЯ

- Первый: Математика; последний: Python; количество: 3.
- Добавь «История» в конец вручную в исходный список: последний и количество должны измениться.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 62 / 2 ИЗ 15 В ТЕМЕ

Список покупок

Начальный

Создать / ex62.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

append() меняет список на месте. Для простого вывода каждый элемент можно печатать внутри for.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
items = ["чай"]
items.append("молоко")
for item in items:
    print(item)
```

ОБРАТИ ВНИМАНИЕ

Не присваивай списку результат append(): этот метод возвращает None.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

В готовом файле дан список ["хлеб", "яблоки"]. Запроси один непустой товар, добавь его в конец через append(). Выведи весь список по одному товару на строке и общее количество товаров.

ПРОВЕРЬ СЕБЯ

- Ввод «сыр» хлеб, яблоки, сыр; количество 3.
- Ввод «хлеб» тоже добавляется: повторения в этой задаче разрешены.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 63 / 3 ИЗ 15 В ТЕМЕ

Одна коробка или две?

Начальный

Прочитать / ex63.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Два имени могут ссылаться на один список. Изменение через любое имя тогда видно через оба. Копия создаёт другой список.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = [1, 2]
b = a.copy()
b.append(3)
print(a)  # [1, 2]
```

ОБРАТИ ВНИМАНИЕ

Знак = сам по себе не означает копирование.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Предскажи оба вывода. Нарисуй, какие имена указывают на один объект. Затем замени b = a на b = a.copy() и предскажи новый результат до запуска.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
a = [10, 20]
b = a
b[0] = 99
a.append(5)
print(a)
print(b)
```

ПРОВЕРЬ СЕБЯ

- Запиши два прогноза для каждого из двух вариантов.
- Объясни, почему строка a.append(5) тоже влияет или не влияет на b.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 64 / 4 ИЗ 15 В ТЕМЕ

Вводим оценки

Начальный

Создать / ex64.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Пустой список [] удобно постепенно заполнять. В цикле каждое введённое число добавляется через append().

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
numbers = []
for i in range(2):
    numbers.append(int(input("Число: ")))
print(numbers)
```

ОБРАТИ ВНИМАНИЕ

Создай список до цикла, чтобы прежние оценки не исчезали.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Запроси количество оценок n от 0 до 10. Введи n целых оценок от 0 до 20 и сохрани их в списке. Затем выведи список в стандартном виде Python и число его элементов.

ПРОВЕРЬ СЕБЯ

- 3; затем 12, 15, 9 [12, 15, 9]; количество 3.
- 0 []; количество 0, без запросов оценок.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 65 / 5 ИЗ 15 В ТЕМЕ

Среднее пустого списка

Начальный Исправить / ex65.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

У пустого списка длина 0. Среднее sum(items) / len(items) нельзя вычислять без проверки, иначе возникнет деление на ноль.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
values = [2, 4]
if len(values) > 0:
    print(sum(values) / len(values))  # 3.0
```

ОБРАТИ ВНИМАНИЕ

Нулевая оценка внутри списка не делает список пустым.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь код так, чтобы для пустого списка выводилось «Нет оценок», а для непустого - среднее с двумя знаками. Добавь комментарий о причине ошибки.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
marks = []
average = sum(marks) / len(marks)
print(average)
```

ПРОВЕРЬ СЕБЯ

- [] Нет оценок.
- [12, 14, 16] 14.00; [0] 0.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 66 / 6 ИЗ 15 В ТЕМЕ

Только положительные

Средний Создать / ex66.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Фильтр создаёт новый список только из подходящих элементов. Исходный список можно сохранить целиком для сравнения.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
values = [1, 4, 7]
even = []
for value in values:
    if value % 2 == 0:
        even.append(value)
print(even)  # [4]
```

ОБРАТИ ВНИМАНИЕ

Не удаляй элементы исходного списка во время его перебора.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Для списка целых values создай новый список всех положительных чисел в исходном порядке. Ноль не включай. Исходный список не меняй; выведи оба списка. Проверяй на готовых списках, ввод не обязателен.

ПРОВЕРЬ СЕБЯ

- [-2, 0, 5, 3, -1] [5, 3].
- [] и [-3, 0] []; [2, 2] [2, 2].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 67 / 7 ИЗ 15 В ТЕМЕ

Минимум и максимум вручную

Средний

Создать / ex67.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Для непустого списка начальным кандидатом удобно сделать первый элемент. Каждый следующий элемент сравнивается с текущими крайними значениями.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
values = [6, 2, 9]
smallest = values[0]
for value in values:
    if value < smallest:
        smallest = value
print(smallest)  # 2
```

ОБРАТИ ВНИМАНИЕ

Если начать максимум с 0, список из отрицательных чисел даст неверный ответ.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Для непустого списка целых найди минимум, максимум и разность между ними. Используй цикл и условия; min(), max() и сортировка запрещены.

ПРОВЕРЬ СЕБЯ

- [4, -2, 9, 0] минимум -2, максимум 9, разность 11.
- [-5, -3] -5, -3, 2; [7] 7, 7, 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 68 / 8 ИЗ 15 В ТЕМЕ

Все позиции числа

Средний

Создать / ex68.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

range(len(items)) перебирает допустимые индексы. По индексу можно проверить значение и сохранить номер найденного совпадения.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
values = [4, 8, 4]
for i in range(len(values)):
    if values[i] == 8:
        print(i)  # 1
```

ОБРАТИ ВНИМАНИЕ

После первого совпадения поиск не заканчивается.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Даны список целых и искомое число. Создай список всех индексов, на которых оно встречается. Если совпадений нет, выведи «Не найдено», иначе список индексов. Индексы отсчитываются с 0.

ПРОВЕРЬ СЕБЯ

- [2, 7, 2, 2], искомое 2 [0, 2, 3].
- [1, 3], искомое 2 и пустой список Не найдено.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 69 / 9 ИЗ 15 В ТЕМЕ

Копия испортила оригинал

Средний

Исправить / ex69.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Для списка чисел copy() создаёт отдельный контейнер. После этого замена элемента копии не меняет исходный список.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
base = [3, 6]
extra = base.copy()
extra[1] = 0
print(base)  # [3, 6]
```

ОБРАТИ ВНИМАНИЕ

В этой задаче все элементы - числа; вложенных списков нет.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Требуется получить изменённую копию оценок и оставить исходные оценки прежними. Исправь одну строку. Объясни, почему исходная запись создала общее состояние.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
original = [10, 12, 14]
changed = original
changed[0] = 20
print(original)
print(changed)
```

ПРОВЕРЬ СЕБЯ

- Исходный список должен быть [10, 12, 14].
- Копия должна быть [20, 12, 14]; добавление в копию тоже не меняет оригинал.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 70 / 10 ИЗ 15 В ТЕМЕ

sort() или sorted()?

Средний

Исправить / ex70.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

sort() меняет исходный список и возвращает None. sorted() возвращает новый отсортированный список, сохраняя исходный.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = [5, 1]
b = sorted(a)
print(a)  # [5, 1]
print(b)  # [1, 5]
```

ОБРАТИ ВНИМАНИЕ

Присваивание результата sort() не сохраняет отсортированные элементы.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Программа должна показать исходные времена и отдельно отсортированную копию. Сейчас она меняет исходник и печатает None. Исправь код, затем объясни различие sort() и sorted().

КОД С ОШИБКОЙ: ИСПРАВЬ

```
times = [12, 5, 9]
ordered = times.sort()
print(times)
print(ordered)
```

ПРОВЕРЬ СЕБЯ

- Исходный: [12, 5, 9]; сортированный: [5, 9, 12].
- Для [] обе версии пустые; для [4, 4, 1] копия [1, 4, 4].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 71 / 11 ИЗ 15 В ТЕМЕ

Без повторений, но по порядку

Сложный Создать / ex71.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Проверка x in result позволяет узнать, добавляли ли значение раньше. Сохраняя первое появление, можно удалить повторы без сортировки.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
result = []
for value in [1, 1, 3]:
    if value not in result:
        result.append(value)
print(result)  # [1, 3]
```

ОБРАТИ ВНИМАНИЕ

Сортировка изменит исходный порядок и не соответствует условию.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Дан список названий городов. Создай новый список без повторений, сохраняя порядок первого появления. Регистр учитывается: «Льеж» и «льеж» разные записи. set() использовать нельзя.

ПРОВЕРЬ СЕБЯ

- ["Льеж", "Брюссель", "Льеж", "Намюр"] ["Льеж", "Брюссель", "Намюр"].
- [] []; ["A", "A", "a"] ["A", "a"].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 72 / 12 ИЗ 15 В ТЕМЕ

Сдвиг списка вправо

Сложный

Создать / ex72.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Срезы списка дают новые списки. Оператор + соединяет их. Для циклического сдвига полезно сначала уменьшить большой сдвиг по модулю длины.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
items = ["A", "B", "C"]
moved = items[1:] + items[:1]
print(moved)  # ['B', 'C', 'A']
```

ОБРАТИ ВНИМАНИЕ

Модуль по длине пустого списка вызывает деление на ноль; обработай пустоту первым делом.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Даны список и целое k >= 0. Создай новый список после циклического сдвига вправо на k позиций. Исходник не меняй. Пустой список даёт []. Можно использовать срезы.

ПРОВЕРЬ СЕБЯ

- [1, 2, 3, 4], k = 1 [4, 1, 2, 3]; k = 6 [3, 4, 1, 2].
- k = 0 исходный порядок; [], k = 10 [].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 73 / 13 ИЗ 15 В ТЕМЕ

Гистограмма оценок

Сложный

Создать / ex73.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Список счётчиков может хранить частоту каждого возможного значения. Если значения от 0 до 5, достаточно шести ячеек.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
counts = [0] * 3
for value in [0, 2, 2]:
    counts[value] += 1
print(counts)  # [1, 0, 2]
```

ОБРАТИ ВНИМАНИЕ

Вывод должен идти по оценкам 0, 1, 2, 3, 4, 5, а не по порядку входа.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Дан список целых оценок от 0 до 5. Создай шесть счётчиков и выведи для каждой оценки строку «оценка: столько звёздочек», включая оценки с нулевой частотой. dict() и count() запрещены: частоты накопи одним проходом.

ПРОВЕРЬ СЕБЯ

- [5, 3, 5, 0, 3, 3] частоты [1, 0, 0, 3, 0, 2].
- [] шесть строк без звёздочек.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 74 / 14 ИЗ 15 В ТЕМЕ

Слияние двух очередей

Сложный

Создать / ex74.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Два индекса могут независимо следить за текущими элементами разных списков. В результат переносится меньший текущий элемент; затем двигается только его индекс.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
a = [2, 8]
b = [4, 9]
print(a[0] < b[0])  # True
```

ОБРАТИ ВНИМАНИЕ

Проверяй границы обоих индексов до чтения элементов.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Даны два списка целых, уже отсортированных по возрастанию. Слей их в новый отсортированный список, сохраняя повторы. sorted(), sort() и повторная сортировка запрещены. Когда один список закончится, добавь оставшиеся элементы другого.

ПРОВЕРЬ СЕБЯ

- [1, 4, 7] и [2, 4, 8] [1, 2, 4, 4, 7, 8].
- [] и [3, 5] [3, 5]; два пустых [].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 75 / 15 ИЗ 15 В ТЕМЕ

Корзина школьной ярмарки

Сложный

Создать / ex75.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

В параллельных списках одинаковый индекс связывает название, цену и количество одной позиции. Условие должно гарантировать равную длину списков.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
names = ["чай", "вода"]
prices = [2.0, 1.0]
for i in range(len(names)):
    print(names[i], prices[i])
```

ОБРАТИ ВНИМАНИЕ

Товар с нулевым количеством не выводится, но индексы остальных списков сохраняют связь.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Даны равные по длине списки names, prices и quantities: названия, цены > 0, целые количества >= 0. Выведи строки только для ненулевых количеств и общую стоимость. От 50 евро скидка 10% на всю корзину. Покажи сумму до скидки, скидку и итог с двумя знаками. Исходные списки не меняй.

ПРОВЕРЬ СЕБЯ

- Имена A/B, цены [10, 15], количества [2, 2] 50.00; скидка 5.00; итог 45.00.
- Цены [4, 3], количества [0, 2] только вторая позиция; итог 6.00. Пустая корзина 0.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ТЕМА 6 / ЗАДАНИЯ 76-90

Функции и сборка программ

Выделяй повторяемые действия и проверяй их отдельно.

- Функция - именованный блок кода. def объявляет функцию, но не запускает её. Вызов name(...) выполняет тело. Параметр - имя внутри объявления, аргумент - переданное при вызове значение.
- return возвращает значение и завершает вызов. print() только показывает текст. Без выполненного return функция возвращает None. Для проверки результата можно написать assert expression: ложное выражение вызовет AssertionError.
- Имена, созданные внутри функции, обычно локальны. Передавай данные параметрами, а результат возвращай через return. В задачах этого раздела ввод и вывод размещай снаружи функций, если не сказано иначе.
- Значение параметра по умолчанию позволяет не передавать этот аргумент. Не ставь изменяемый список как обычное значение по умолчанию. Для нескольких результатов используй return a, b: получится кортеж, который можно распаковать в два имени.

ОСНОВНОЙ СИНТАКСИС

```
def rectangle_area(width, height):
    return width * height

area = rectangle_area(3, 4)
print(area)  # 12
assert rectangle_area(2, 5) == 10
```

ЧТО ПРИГОДИТСЯ

def name(x):	Объявить функцию с параметром
return result	Вернуть результат и закончить вызов
name(5)	Вызвать функцию с аргументом
def name(x, step=1):	Задать значение по умолчанию
assert result == expected	Проверить ожидаемый результат

ЧАСТЫЕ ОШИБКИ

Не путай вызов и объявление. return внутри цикла может завершить функцию после первого элемента. Не используй случайные глобальные переменные вместо параметров.

ЗАДАНИЕ 76 / 1 ИЗ 15 В ТЕМЕ

Площадь квадрата

Начальный Создать / ex76.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Функция может получать число и возвращать вычисленный результат. Вызывающий код решает, печатать его или использовать дальше.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def triple(number):
    return number * 3
print(triple(4))  # 12
```

ОБРАТИ ВНИМАНИЕ

Функция должна вернуть число, а не строку «Площадь: ...».

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши square_area(side): получает сторону >= 0 и возвращает площадь квадрата. Внутри не должно быть input() и print(). Снаружи вызови её для трёх разных сторон и выведи результаты.

ПРОВЕРЬ СЕБЯ

- square_area(4) 16; square_area(0) 0.
- square_area(2.5) 6.25; проверь также assert.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 77 / 2 ИЗ 15 В ТЕМЕ

Порядок аргументов

Начальный

Прочитать / ex77.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Позиционные аргументы связываются с параметрами по порядку. Имена при вызове позволяют явно указать, какому параметру предназначено значение.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def subtract(a, b):
    return a - b
print(subtract(b=2, a=7))  # 5
```

ОБРАТИ ВНИМАНИЕ

Имя параметра при вызове важнее позиции именованного аргумента.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Предскажи три вывода и объясни, какие значения получают price и count в каждом вызове. Затем проверь запуском. В качестве собственного примера придумай вызов с именованными аргументами в другом порядке.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
def total(price, count=1):
    return price * count
print(total(4, 3))
print(total(count=2, price=5))
print(total(7))
```

ПРОВЕРЬ СЕБЯ

- Для каждого вызова запиши отдельную пару значений параметров.
- В последнем вызове учитывай значение count по умолчанию.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 78 / 3 ИЗ 15 В ТЕМЕ

Печатает, но не возвращает

Начальный Исправить / ex78.py Основное

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

print() и return решают разные задачи. Если результат функции нужен для следующего вычисления, его необходимо вернуть.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
1  def double(x):
2      return x * 2
3  value = double(3)
4  print(value + 1)  # 7
```

ОБРАТИ ВНИМАНИЕ

Если написать return print(...), вернётся результат print(), то есть None.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь функцию, чтобы result содержал сумму и строка result + 1 работала. Не добавляй вывод внутри функции. Объясни, откуда в исходном варианте взялось None.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
1  def add(a, b):
2      print(a + b)
3  result = add(2, 3)
4  print(result + 1)
```

ПРОВЕРЬ СЕБЯ

- add(2, 3) 5; итоговый print() 6.
- add(-2, 2) 0.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 79 / 4 ИЗ 15 В ТЕМЕ

Повтор строки

Начальный

Создать / ex79.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Параметр по умолчанию используется, когда соответствующий аргумент пропущен. Сначала идут обязательные параметры, затем необязательные.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def add_step(x, step=1):
    return x + step
print(add_step(5))     # 6
print(add_step(5, 3))  # 8
```

ОБРАТИ ВНИМАНИЕ

Не печатай строку внутри функции; результат должен быть доступен вызывающему коду.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши repeat_text(text, times=2), возвращающую строку text, повторённую times раз без разделителя. times - целое >= 0. Покажи вызов с одним аргументом и с двумя.

ПРОВЕРЬ СЕБЯ

- repeat_text("xa") xaxa.
- repeat_text("ab", 3) ababab; repeat_text("x", 0) пустая строка.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 80 / 5 ИЗ 15 В ТЕМЕ

Куда пропало число?

Начальный

Вопрос / ex80.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Одинаковые имена снаружи и внутри функции могут обозначать разные локальные переменные. Возвращённое значение изменит внешний объект только если вызывающий код его использует соответствующим образом.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def make_number():
    value = 7
    return value
result = make_number()
print(result)  # 7
```

ОБРАТИ ВНИМАНИЕ

Внутреннее присваивание number = 9 не записывает в одноимённую внешнюю переменную.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Предскажи вывод. Объясни, почему внешнее number не стало другим. Затем измени только строку вызова так, чтобы во внешнее number попал возвращённый результат. global использовать нельзя.

КОД ЗАДАНИЯ: СНАЧАЛА ПРОЧИТАЙ

```
number = 4
def get_number():
    number = 9
    return number
get_number()
print(number)
```

ПРОВЕРЬ СЕБЯ

- Запиши прогноз для исходного и изменённого вариантов.
- Объясни роль присваивания в строке вызова.

ЗАДАНИЕ 81 / 6 ИЗ 15 В ТЕМЕ

Цена с налогом

Средний

Создать / ex81.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Формула внутри функции может зависеть от нескольких параметров. В учебном расчёте лучше вернуть число, а нужный формат задать только при выводе.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def add_percent(value, percent):
    return value * (1 + percent / 100)
print(f"{add_percent(50, 10):.2f}")  # 55.00
```

ОБРАТИ ВНИМАНИЕ

Это учебная формула; реальные правила налогов в задании не рассматриваются.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши price_with_tax(price, rate=21): цена >= 0, ставка от 0 до 100. Возвращай цену после добавления указанного процента. Внешний код выводит два знака после точки. Ставки заданы для тренировки вычислений.

ПРОВЕРЬ СЕБЯ

- price_with_tax(100) 121.00 при выводе.
- price_with_tax(50, 10) 55.00; price_with_tax(0) 0.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 82 / 7 ИЗ 15 В ТЕМЕ

Год как логическая функция

Средний

Создать / ex82.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Функция может возвращать bool. Тогда результат удобно прямо использовать в if, без сравнения с текстом.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def is_even(n):
    return n % 2 == 0
if is_even(8):
    print("Чётное")
```

ОБРАТИ ВНИМАНИЕ

Возвращай True/False, а не строки «True»/«False».

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши is_leap(year), возвращающую True для високосного года и False для обычного. year >= 1. Применяй правило кратности 4, исключения 100 и восстановления 400. Снаружи выведи словесный ответ.

ПРОВЕРЬ СЕБЯ

- assert is_leap(2024); assert is_leap(2000).
- assert not is_leap(1900); assert not is_leap(2026).

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 83 / 8 ИЗ 15 В ТЕМЕ

Три результата статистики

Средний

Создать / ex83.py

Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

return a, b, c возвращает кортеж из трёх элементов. Его можно распаковать: x, y, z = function(). Число имён должно совпадать с числом элементов.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def neighbours(n):
    return n - 1, n + 1
left, right = neighbours(5)
print(left, right)  # 4 6
```

ОБРАТИ ВНИМАНИЕ

В этой задаче пустой список не допускается; это часть договора функции.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши statistics(values) для непустого списка чисел. Верни минимум, максимум и среднее в этом порядке. Можно применять min(), max() и sum(). Снаружи распакуй ответ в три переменные и выведи их; среднее с двумя знаками.

ПРОВЕРЬ СЕБЯ

- [12, 14, 16] 12, 16, 14.00.
- [-5, -1] -5, -1, -3.00; [7] 7, 7, 7.00.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 84 / 9 ИЗ 15 В ТЕМЕ

Общий список по умолчанию

Средний

Исправить / ex84.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

Значение параметра по умолчанию создаётся при объявлении функции, а не при каждом вызове. Для свежего списка используй None и создавай [] внутри при необходимости.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def fresh(items=None):
    if items is None:
        items = []
    return items
```

ОБРАТИ ВНИМАНИЕ

Проверяй items is None, а не «if not items»: явно переданный пустой список тоже должен сохраняться.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Исправь функцию: вызовы без второго аргумента должны получать новый список каждый раз. Если список передан явно, добавление должно изменить именно его. Объясни, почему исходные два вызова без списка делили данные.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
def add_item(value, items=[]):
    items.append(value)
    return items
print(add_item(1))
print(add_item(2))
```

ПРОВЕРЬ СЕБЯ

- Первый вызов без списка [1]; второй [2].
- Передай existing = [9] и значение 3 existing становится [9, 3].

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 85 / 10 ИЗ 15 В ТЕМЕ

Возврат стоит слишком рано

Средний

Исправить / ex85.py

Исходный код задания уже в файле. Ответы пиши комментариями.

СНАЧАЛА РАЗБЕРИСЬ

return немедленно заканчивает вызов функции. Если он находится внутри цикла, обработка может закончиться после первого элемента.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def total(values):
    result = 0
    for value in values:
        result += value
    return result
```

ОБРАТИ ВНИМАНИЕ

В исходнике пустой список возвращает None, потому что тело цикла ни разу не выполняется.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Функция должна возвращать количество положительных элементов, включая корректный ответ 0 для пустого списка. Исправь положение return и проверь все элементы.

КОД С ОШИБКОЙ: ИСПРАВЬ

```
def count_positive(values):
    count = 0
    for value in values:
        if value > 0:
            count += 1
        return count
```

ПРОВЕРЬ СЕБЯ

- [1, -2, 3, 4] 3.
- [] 0; [-5, -1] 0; [0, 2] 1.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 86 / 11 ИЗ 15 В ТЕМЕ

Простота как функция

Сложный Создать / ex86.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Возвращай False сразу, как только найдено опровержение. True возвращается только после завершения всех нужных проверок.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def has_negative(values):
    for value in values:
        if value < 0:
            return True
    return False
```

ОБРАТИ ВНИМАНИЕ

Квадрат делителя позволяет обойтись без дробного квадратного корня.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши is_prime(n) для целого n >= 0. Верни bool; 0 и 1 не простые. Проверяй делители только пока divisor * divisor <= n. Затем внешним циклом выведи все простые от 2 до 30.

ПРОВЕРЬ СЕБЯ

- is_prime(2) и is_prime(29) True; is_prime(25) False.
- В выводе: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 87 / 12 ИЗ 15 В ТЕМЕ

Учебный логин

Сложный

Создать / ex87.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Результат одной функции можно передать другой. Нормализация текста и проверка данных удобнее читаются как отдельные шаги.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def clean(text):
    return text.strip().lower()
print(clean("  CAT  "))  # cat
```

ОБРАТИ ВНИМАНИЕ

До first[0] обязательно проверь результат очистки.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши clean_name(text), возвращающую очищенный строчный текст. Напиши make_login(first, last): после очистки оба поля должны быть непустыми; тогда верни первую букву имени плюс фамилию. Иначе верни None. Входные непустые поля состоят из латинских букв. Внешний код выводит логин или «Нет данных».

ПРОВЕРЬ СЕБЯ

- " tOM ", " LAMBERT " tlambert.
- "Anna", "Fox" afox; имя из пробелов None.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 88 / 13 ИЗ 15 В ТЕМЕ

Фильтр с порогом

Сложный

Создать / ex88.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Функция, создающая новый список, может обходиться без изменения аргумента. Независимость результата полезно проверить последующим изменением этого результата.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def copy_all(values):
    result = []
    for value in values:
        result.append(value)
    return result
```

ОБРАТИ ВНИМАНИЕ

Строго больше исключает само пороговое значение.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши above_threshold(values, limit), возвращающую новый список чисел строго больше limit в исходном порядке. Затем напиши average_or_none(values): среднее непустого списка или None. Применяй вторую функцию к результату первой.

ПРОВЕРЬ СЕБЯ

- [3, 8, 5, 8], limit 5 [8, 8]; среднее 8.
- [], limit 0 []; среднее None. Исходный список после вызовов тот же.

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 89 / 14 ИЗ 15 В ТЕМЕ

Библиотека расчётов времени

Сложный Создать / ex89.py

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Несколько небольших функций можно собрать в цепочку: каждая отвечает за один понятный этап и имеет собственные проверки.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def to_seconds(minutes, seconds):
    return minutes * 60 + seconds
assert to_seconds(2, 5) == 125
```

ОБРАТИ ВНИМАНИЕ

Функции вычислений не должны сами запрашивать ввод или печатать.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Напиши to_minutes(h, m), trip_duration(dh, dm, ah, am) и split_duration(minutes). Первая возвращает минуты от полуночи, вторая - путь меньше 24 часов с переходом через полночь, третья - пару часов и минут. Часы 0..23, минуты 0..59; одинаковые времена означают 0. Вторая функция должна вызывать первую.

ПРОВЕРЬ СЕБЯ

- 14:35 17:10 длительность 155 минут (2, 35).
- 23:40 01:15 95 (1, 35); 08:00 08:00 (0, 0).

ТВОЙ ПЛАН / ЗАМЕТКИ

ЗАДАНИЕ 90 / 15 ИЗ 15 В ТЕМЕ

Итоговый отчёт по оценкам

Сложный Создать / ex90.py Основное

Данные уже в файле. Обработку и вывод допиши самостоятельно.

СНАЧАЛА РАЗБЕРИСЬ

Большую задачу разделяй по ответственности: проверить данные, выбрать подходящие значения, посчитать статистику, оформить вывод. Проверяй каждый этап отдельно.

ПРИМЕР НА ДРУГОЙ СИТУАЦИИ

```
def is_allowed(value):
    return 0 <= value <= 5
assert is_allowed(3)
assert not is_allowed(8)
```

ОБРАТИ ВНИМАНИЕ

Неверные оценки не должны влиять ни на среднее, ни на число зачётов.

ТЕПЕРЬ ТВОЯ ЗАДАЧА

Дан список целых оценок, где встречаются неверные значения. Напиши valid_marks(values): новый список только из 0..20; passed_count(values): количество >= 10; average_or_none(values): среднее или None. Внешний код выводит число отброшенных, число допустимых, число зачётов и среднее с двумя знаками либо «Нет оценок». Зачёты и среднее считай только по допустимым.

ПРОВЕРЬ СЕБЯ

- [-1, 12, 8, 21, 20] отброшено 2; допустимых 3; зачётов 2; среднее 13.33.
- [] 0, 0, 0, Нет оценок; [25, -3] 2, 0, 0, Нет оценок.

ТВОЙ ПЛАН / ЗАМЕТКИ

Всё под рукой

Ввод целого	n = int(input("Число: "))
Ввод дробного	x = float(input("Цена: "))
Текст с числом	print(f"Итог: {total:.2f}")
Целые группы и остаток	groups = n // size; rest = n % size
Включительный диапазон	low <= value <= high
Выбор	if ...: / elif ...: / else:
Повторение n раз	for i in range(n):
Числа от 1 до n	for i in range(1, n + 1):
Цикл до события	while condition:
Выход / пропуск	break / continue
Очистить текст	clean = text.strip().lower()
Последние 4 символа	last = text[-4:]
Развернуть строку	backwards = text[::-1]
Новый список	result = []; result.append(value)
Перебор списка	for value in values:
Копия / сортированная копия	copy = values.copy(); ordered = sorted(values)
Функция	def name(parameter): ... return result
Проверка результата	assert name(value) == expected

КАК ПОЛЬЗОВАТЬСЯ ПАМЯТКОЙ

Строки в этой таблице - напоминания, а не одна готовая программа. Многоточие в схеме функции означает пропущенное тело; в своём решении напиши нужные инструкции и отступы.

Ошибка подсказывает путь

Посмотри последнюю строку сообщения: там вид ошибки и причина. Затем найди свой файл и номер строки выше. Номер показывает, где ошибка обнаружена; причина иногда находится раньше.

SyntaxError	Проверь двоеточия, скобки и кавычки. Незакрытая строка может мешать чтению следующих строк.
IndentationError	Проверь отступы тела if, for, while и def. Используй четыре пробела, не смешивай пробелы и табуляции.
NameError	Имя не создано или написано иначе: total и Total различаются.
TypeError	Операция не подходит типам: например, текст складывается с числом без преобразования.
ValueError	Преобразование не удалось: int("кот") не является целой записью.
ZeroDivisionError	Деление или остаток по модулю с нулевым делителем.
IndexError	Индекс за пределами строки или списка; возможно, данные пустые.
AssertionError	Проверка assert не совпала с ожидаемым результатом.

- Сведи вход к маленькому примеру и повтори проблему.
- До подозрительной строки временно выведи значения и type(...).
- Исправь одну причину, снова запусти программу и сравни результат.
- После исправления повтори обычный и граничный случаи; убери временный вывод.

КОГДА СООБЩЕНИЕ НЕ ПОЯВИЛОСЬ

Если ошибок нет, но ответ неверный, это ошибка логики. Проверь порядок действий, границы диапазона и место обновления переменных. Если программа не останавливается, прерви Ctrl+C и проверь условие while.

Отмечай движение

Поставь отметку в клетке, когда можешь объяснить решение и проверил предложенные случаи. Обведи номер, если хочешь вернуться к задаче позже.

1. ВВОД, ПЕРЕМЕННЫЕ И ВЫЧИСЛЕНИЯ

01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

2. УСЛОВИЯ: IF, ELIF, ELSE

16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

3. ЦИКЛЫ: FOR И WHILE

31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

4. СТРОКИ И ОБРАБОТКА ТЕКСТА

46	47	48	49	50	51	52	53	54	55	56	57	58	59	60
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

5. СПИСКИ И НАБОРЫ ДАННЫХ

61	62	63	64	65	66	67	68	69	70	71	72	73	74	75
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

6. ФУНКЦИИ И СБОРКА ПРОГРАММ

76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Дата / что ещё повторить: _____

Жёлтый - начальный уровень, мятный - средний, розовый - сложный. Один полностью понятный разбор полезнее нескольких решений, которые ты не можешь объяснить.

Откуда взялась структура

За образец взят предоставленный сборник «C01 · Les bases de PHP - exercices», Nicolas Hinckxt, 2026-2027: 12 страниц, упражнения 01-31. Из него сохранены четыре формата работы, три уровня, задания на чтение и исправление кода, проверка других значений и бытовые расчёты.

Это самостоятельная русская адаптация для Python. Сюжеты про оценки, время, размен, цифры числа, палиндромы и парковку переработаны под Python; разделы условий, циклов, списков и функций расширены новыми упражнениями. Требования исходника к PHP, серверу и браузеру здесь не применяются.

ПРОВЕРКА ЯЗЫКА И ДАЛЬНЕЙШЕЕ ЧТЕНИЕ

Ввод, числа и строки
https://docs.python.org/3/tutorial/introduction.html

Условия, циклы и функции
https://docs.python.org/3/tutorial/controlflow.html

Списки и структуры данных
https://docs.python.org/3/tutorial/datastructures.html

Встроенные функции
https://docs.python.org/3/library/functions.html

Методы строк и типы
https://docs.python.org/3/library/stdtypes.html

Запуск интерпретатора
https://docs.python.org/3/tutorial/interpreter.html

Работа с Python в VS Code
https://code.visualstudio.com/docs/python/python-tutorial

ЧТО ПРОВЕРЕНО

Все 90 мини-примеров и 6 примеров в начале тем выполнены при подготовке пособия. Стартовые файлы соответствуют номерам задач. В заданиях на исправление исходные ошибки намеренно сохранены. Готовых решений нет.

Денежные примеры используют float для тренировки и вывод с двумя знаками. В реальных расчётах денег часто применяют целые минимальные единицы или Decimal. Проверки адресов, паролей и имён здесь ограничены только явно заданными учебными правилами.

После пособия можно переходить к словарям, работе с файлами, try / except, модулям и небольшим проектам. Они не нужны для решения этих 90 задач.